

Eight Defects in the LBNL HVAC Fault-Detection Benchmarks, and the Protocol That Found Them

Yassh Singh

Thompson Rivers University, Kamloops, Canada

Abstract. The LBNL fault-detection datasets are the nearest thing HVAC fault detection has to a common benchmark. We report eight defects, unreported to our knowledge: two families in which one run ships four times under four severity labels, one also mislabelled; one run shipped under two family labels; two columns carrying a per-branch constant that classifies every day of every fault file by provenance with zero error; a file whose calendar is rotated by two days; a fault-free file simulated on a different configuration branch than every fault file; documentation that places every fan-powered-unit fault in the wrong zone; and static-pressure bias files whose recorded shift is one tenth of the label. Each ships with machine evidence and a command. Three were found by a hash gate, the rest by audits under a pre-registration protocol that also voided two of our own baseline runs before they reached print; the four log-generic gates pass on two public process-mining logs. We place the defects against the event-log imperfection patterns and argue that benchmark-level defects need their own taxonomy.

Keywords: data quality · event log imperfection · benchmark leakage · fault detection

1 Introduction

A benchmark rests on a premise that is rarely checked: that the files are what their names say. In building fault detection and diagnostics (FDD) the evaluation protocols grade an algorithm against seeded faults of known type and severity [5, 14], so a mislabelled severity or a duplicated run does not add noise; it redefines the quantity scored. And the field rarely re-runs anything: a preprint audit of sixty-five machine-learning HVAC FDD studies found two with a code link, one of them broken [10]. A defect in shared data propagates.

Process mining has treated data quality as a stage of its own for a decade: Bose and colleagues on treating log quality before analysis [1], Suriadi and colleagues' eleven imperfection patterns [12], and Weytjens and De Weerd on how easily predictive-monitoring benchmarks leak across their splits [13]. That literature has had few occasions to address a leak that is a property of the corpus rather than the split: a class carried by a few files, where any per-file constant is a perfect feature. That is the case here.

The LBNL FDD datasets [7, 6] are simulated annual one-minute data with seeded faults (their creators’ lessons-learned account [3] does not report the defects below), released CC BY 4.0 under DOI 10.25984/1881324.¹ We use five systems: the single-duct and dual-duct air handlers (SDAHU, 21 annual CSVs; DDAHU, 56), the parallel and series fan-powered units (PFPU and SFPU, 31 each, with Brick semantic models) and the fan coil unit (FCU, 49). This paper reports eight defects found while preparing a detection study whose process-mining results are in a companion paper [11] (its settled finding: process mining helps to get detection but it is not detecting on its own): three by a battery of mechanical gates, the rest by audits of our own work and of the documentation.

2 The Gate Battery

Six checks run against the raw files before any science, none requiring insight; a seventh sits on the detector side. *MD5 over raw and converted files*, grouped, so that byte-identical files under different names surface immediately; this produced E1, E2 and E6. *Monotonicity* of timestamps, counting wrap points rather than reporting a boolean. *Calendar identity*: each fault file’s timestamp set compared with its fault-free sibling’s. *Raw-file rotation*: monotonicity and set identity tested together on the raw CSV, before the conversion step that sorts by timestamp and would mask a rotation. *Brick/TTL coverage*: declared points against columns, by name, because a mapping layer that fails open drops points silently. *Log hashing* of the derived event logs, so that two scenarios reduced to identical logs cannot count as independent evidence.² A *healthy-silence* gate confirms that a rule set is silent on the fault-free year before it may score anything.

None of the gates was designed in advance; each was added after a defect got past the previous set, and the battery is governed by one rule: a lesson is not closed until a test enforces it. Four of the six gates make no reference to HVAC. To show they are log-generic we ran them unchanged over two public process-mining event logs — the Sepsis Cases log [9] and the CoSeLoG receipt-phase log [2] — via pm4py, in seconds. Both pass; the calendar gate reports Sepsis’s ten known midnight-exact stamps. Clean results on curated logs are expected; the point is the cost.

Passing on clean logs shows the gates run, not that they catch anything, so we injected each defect class into the Sepsis log and re-ran the four gates:³ a duplicated file fires the hash gate alone, a case copied under a new identifier the trace-hash gate alone, a rotated case monotonicity alone, a case moved forty years the calendar gate alone. A per-case constant encoding the case’s outcome

¹ <https://data.openei.org/submissions/5763>, also indexed at <https://faultdetection.lbl.gov/data/>.

² `log_hash` per `scenario` and `event_identical_groups` in `outputs/benchmark_v6/*.json`.

³ `outputs/gates_public_log.json`, `gates_injection.json`, `scripts/gates_public_log.py`, `gates_injection.py`.

— the log-level analogue of E3 — fires none of the four: no gate that reads one file at a time can see a relation between an instance and its label (Section 6).

3 The Protocol Around the Gates

The gates are the mechanical part of a protocol whose other parts found the five defects the gates could not. Every experiment is pre-registered: hypotheses, predictions and the falsifiers that would kill them are committed to git before the run (twenty-eight such plans in the current series), and a fired falsifier is published as a result, not buried. A number is quoted only after an audit that recomputes it from the raw files; an audit that finds a defect voids the run; its artefacts are kept under a run-number suffix and never quoted, and the repair is an amendment to the pre-registration written before the re-run. A regression gate (`scripts/regress.py`) regenerates every live artefact from the current code and fails on any value that differs from the committed copy, holding closed experiments frozen at their closing commits; 154 guard tests pin the published numbers and the fired falsifiers, so a change that would alter either fails the build.

What this buys is shown best by the runs it voided. The comparison against a Guideline 36 rule battery (X38) was executed three times: two hostile audits voided the first run for grading terminal units with air-handler rules and feeding E3’s constant into a static-pressure rule; two more voided the second, of the repaired adapter, for a dead constant fan-command column that made a full-speed condition always true, a night-cycle state graded as occupied, and a pooled significance gate that is a tautology once a battery flags every healthy day. Each defect was ours, not the library’s; only the third execution is quoted, with its own falsifier fired on every system. The diagnosis layer (X37) took three runs, two script defects found by reading the rows against the pre-registered wording and fixed to that wording rather than re-tuned. The errata and the voided runs are this protocol’s evidence of rigour: none of the defects reached print, every quoted number regenerates, and what the framework detects stands.

4 The Eight Defects

Table 1 summarises; “healthy” means the fault-free file; reproducing commands are in `ERRATA.md`.

4.1 E1 and E2 — two families, one run each

The four files `oa_bias_-4/-2/2/4_annual.csv` are byte-identical: one raw MD5 (`89b13704...`) and one parquet MD5 (`63b857d7...`) across all four. So are `coi_leakage_010/025/040/050` (`a9fdcf50...`, `5133132d...`). The 21-file SDAHU set contains 15 distinct files: the healthy year plus 14 fault scenarios; severity-response analysis for either family is not underpowered but impossible.

Table 1. Eight machine-verified defects. Commands: `ERRATA.md`; evidence: `outputs/ week0_audit*`, `e3_leakage`, `e5_schedule`, `x11_branch`, `e7_zone_label`, `e8_bias_magnitude (.json)`.

ID	System	Files / columns and defect	Consequence
E1	SDAHU	<code>oa_bias</code> (4 files): one run shipped four times; the labelled bias is not in the logged column	Four labels index one run; it is a fault run, not a healthy negative
E2	SDAHU	<code>coi_leakage</code> (4 files): one run shipped four times	The severity ladder is vacuous: one scenario, not four
E3	SDAHU	<code>SA_SP</code> , <code>SA_SPSPT</code> : a constant ≈ 401.86 added to one column in the healthy branch and subtracted from the other in the fault branch	Either column classifies every day by file provenance with zero error
E4	SFPU	<code>SensorBias_RMTEMP_-2C</code> : raw file rotated by exactly 2,880 rows	Positional alignment misaligns a year; scenario excluded, 29 scored
E5	SDAHU	healthy vs. all 20 fault files: different configuration branch (damper floor 0.000 vs 0.100, a one-hour schedule shift, a -1.06°F offset)	Healthy-vs-fault divergence carries a branch offset; one of our own detections was provenance
E6	FCU	<code>Fouling_{Cooling,Heating}_Airside_Minor</code> : one run under two <i>family</i> labels (one MD5)	One label is wrong, the data cannot say which; scored once
E7	PFPU, SFPU	every fault file: the documentation places every seeded fault in the west zone (<code>_W</code>); only the south zone's columns (<code>_S</code>) move (57 of 60; three indeterminate)	A reader scoring localisation against the documented zone scores every correct alarm as wrong
E8	DDAHU	static-pressure bias files: the deck static column is unchanged; the four mixing-box statics shift by +0.20 and +0.40 in.wg for the -2 and -4 in.wg labels (the $+2^\circ\text{C}$ file shifts by exactly its label)	Severity ladders built from these labels are off by ten

`oa_bias` is also mislabelled. The recorded `OA_TEMP` tracks the healthy file's weather to within 0.33°F (mean 0.02°F , signed mean ≈ 0 , 38% exact zeros) — nowhere near the labelled $\pm 2\text{--}4^\circ\text{F}$ — and is bit-identical across all fault files, so the labelled sensor-side bias is absent from the data. Yet the run misbehaves relative to the other families, consistently with a bias on the controller's outdoor-air input rather than on the logged sensor, though indistinguishable from a mislabelled cooling-lockout setpoint. What is proven is narrower and sufficient: the labelled bias is not in the column, the run is faulty, and the four labels are fiction. Published work has trained an outdoor-air-bias class on these files [4]; we assert only that the bias it names is not in the data.

How they were found. The hash gate produced both duplicate groups in one pass. The mislabel was overturned twice: the files were first taken for an independent healthy year on the strength of the matching weather column, until a blind residual channel “false-alarmed” on them at 48.8%; a later audit (E5) attributed most of the divergence from the healthy file to the branch offset.

Consequence and handling. After the relabel SDAHU has no independent healthy negative run at all, a limitation every SDAHU study inherits; both families are single scenarios in our configuration and `oa_bias` is never used as a negative.

4.2 E3 — a per-branch constant in two columns

In the healthy file `SA_SPSPT` is constant at 1.60746 and `SA_SP` sits near 402; in every one of the twenty fault files `SA_SPSPT` is constant at -400.25253 and `SA_SP` lies between 1.10 and 1.15 in per-file mean of day-medians. An earlier version of this erratum read that as units and roles swapped. It is not: the two branches differ by a single additive constant. $1.60746 - (-400.25253) = 401.85999$, and the healthy file's `SA_SP` with the fan off minus the fault files' with the fan off is again 401.86. Subtracting that constant from the healthy `SA_SP` reproduces the fault files' trace to three decimals; the old unit reading implied a duct at 1.6 inH₂O with the fan off, which is impossible. The healthy branch has the constant added to `SA_SP`; the fault branch has it subtracted from `SA_SPSPT`. Because the offset is branch-specific and E5 establishes that the branches differ, E3 is most likely a symptom of the same divergence, listed separately for its distinct consequence and not counted as independent evidence.

How it was found. By auditing our own PCA baseline (Section 5); the audit gate checks the healthy file against one fault file, and the leakage artefact extends it to all twenty with every file's range. *Handling.* Both columns are in an explicit erratum list that every baseline drops; our detector configurations never mapped them.

4.3 E4 — a date-rotated calendar

The raw file `SFPU_SensorBias_RMTEMP_-2C.csv` is a pure rotation by exactly 2,880 rows, two days at one-minute sampling; its timestamp set is identical to the fault-free file's, the series is non-monotonic with exactly one wrap point, and its first row is 2018-01-03 00:00 where the fault-free file's is 2018-01-01 00:00. The timestamps are correct and the order is not, so a reader that sorts by timestamp never sees it and positional comparison does.

How it was found. By an early audit, not a gate: the monotonicity gate runs on converted files that conversion has already sorted. The raw-file rotation gate was built four phases later so the defect has machine evidence and cannot recur — a gate must sit upstream of the preprocessing that masks it. *Consequence and handling.* Positional comparison against the healthy file misaligns every timestamp for a year; the scenario is excluded, leaving SFPU with 29 scored scenarios, and sort-on-convert plus the calendar-identity gate are permanent.

4.4 E5 — the healthy file is on a different configuration branch

Three independent measurements agree that the SDAHU fault-free file was simulated on a different configuration branch than every fault file. *Damper floor:* during fan-on hours the healthy file's outdoor-air damper minimum is 0.000; every fault file floors at exactly 0.100. *Schedule:* on the pipeline's own occupancy signal the day universe is identical between branches but the healthy file's first occupied minute is 05:01–05:02 on 200 of those days (06:01 on 33 and 06:02 on 70 of the rest) where every fault file's is 06:01 on every day.⁴ An ear-

⁴ `outputs/e5_schedule.json`.

lier version’s occupied-row and runtime figures are retracted. *Simulation-physics offset*: the residual channel’s no-fault baseline differs by -1.06°F between the branches ($+0.008^\circ\text{F}$ on the fault branch, from five concordant measurements, against $+1.071^\circ\text{F}$ on the healthy), uniform across occupied hours and surviving restriction of the healthy file to the fault branch’s schedule.⁵

How it was found. By a hostile audit of E1’s evidence. It had hidden for six phases because every comparison ran against the healthy file and no fault-versus-fault control existed; the standing rule now requires such a control before healthy-versus-fault divergence counts as fault evidence.

Consequence and handling. Every healthy-versus-fault comparison on SDAHU carries the branch offset on top of the fault. Fault-versus-fault divergence is $0.05\text{--}0.14^\circ\text{F}$, an order of magnitude below the offset, so most of E1’s degree-scale divergence belongs to the branch. In our own benchmark the `oa_bias` residual detection sits on the fault-branch baseline ($+0.007$ against $+0.008$): it is branch provenance, and under a pre-registered branch-sensitivity adjudication the SDAHU scorecard is 13 of 14 rather than 14 of 14. The `coi_bias` detections survive by an order of magnitude; the nine rules-carried scenarios are branch-immune, a signature rule comparing position to command within one file.

5 E3 as a Measured Leak

E1, E2, E4 and E6 corrupt an accounting; E3 hands every data-driven model a free discriminator, and we measured how free: one class, healthy, is one file, the other twenty. A threshold chosen from the healthy file alone (the midpoint between its minimum day-median and zero) classifies 365 of 365 healthy days and 7,150 of 7,150 fault-file days correctly on each column: accuracy 1.0, every fault file’s range on the far side.⁶

We know it is exploited in practice because we caught ourselves: our principal-component baseline, fitted on the healthy file and scored on the fault files, reached a squared prediction error of order 10^{37} , a number with no physical reading and the reason the baseline was audited rather than celebrated; the two columns were the entire explanation. This is leakage in Kapoor and Narayanan’s sense [8] — a feature that proxies the label through the data’s construction — but not the split leakage of Weytjens and De Weerd [13]: here the class is confounded with the file, so any column constant within a file is a label.

This is a property of the data, not of any published result; the remedy for SDAHU results that used the full column set is to re-run without the two columns and report both numbers.

⁵ `outputs/x11_branch.json`.

⁶ `outputs/e3_leakage.json`; `scripts/e3_leakage.py`. Healthy SA_SP day-medians span $[401.85, 403.48]$; fault-file day-medians span $[-0.60, 5.97]$.

6 Where the Event-Log Taxonomy Ends

Six of the eight have loose homes in the existing taxonomies: in Bose and colleagues’ grid [1], E4 is an incorrect position and the mislabels of E1, E7 and E8 incorrect activity names; E1, E2 and E6 are corpus-level analogues of Suriadi and colleagues’ synonymous labels [12], at the granularity of files rather than events. E3 and E5 are outside both: a leaked feature and a confounded provenance, properties of how a corpus was assembled rather than of any log within it. A first cut, four classes named by the relation each corrupts: *duplicated instances* (E1, E2, E6) — two labels, one content; *per-instance constants* (E3) — a feature constant within an instance that varies with its label; *confounded provenance* (E5; cf. split leakage [13]) — the partition carries a difference not under study; *misaligned or mislabelled instances* (E4, E7, E8) — the file is not what its name says. The first and fourth are caught by log-generic gates; the second and third cannot be by any gate that reads one file at a time.

7 Recommendations

For maintainers: an errata notice at the distribution points; drop or flag `SA_SP` and `SA_SPSPT`; relabel the duplicated runs, the fan-powered zone and the static-bias magnitudes; republish the rotated file; record the configuration branch; ship checksums. For users (the gates ship as `strata gates LOG.xes`): hash and group files first; test rotation on the raw files; build a fault-versus-fault control into every design; ask of every feature whether it is constant within a file; count scenarios as distinct runs. For reviewers: which columns, and how many distinct files per class, would have caught E3.

8 Disclosure Status

At the time of writing these findings have not yet been reported to the maintainers; they will be submitted through the dataset’s OpenEI contact before camera-ready, and this section will record the date, channel and response.

9 Conclusion

Eight defects in the most widely used public HVAC FDD benchmarks were found not by insight but by mechanical gates and audits run under a pre-registration protocol before and during a detection study. Each would have inflated a published result. All five LBNL archives we audited carry at least one: the single-duct air handler four (E1, E2, E3, E5), the fan-powered units two (E4, E7), the fan coil unit one at the hash gate (E6), and the dual-duct air handler one (E8) that passed every gate and fell to a review of a severity ladder. The same protocol voided two of our own baseline runs before they reached print. The datasets remain the best the field has, which is why a defect in them is a multiplier that no downstream replication will catch; the gates cost a second.

References

1. Bose, R.P.J.C., Mans, R.S., van der Aalst, W.M.P.: Wanna improve process mining results? It's high time we consider data quality issues seriously. In: IEEE Symposium on Computational Intelligence and Data Mining (CIDM). pp. 127–134 (2013). <https://doi.org/10.1109/CIDM.2013.6597225>
2. Buijs, J.C.A.M.: Receipt phase of an environmental permit application process (WABO), CoSeLoG project. 4TU.ResearchData, dataset (2014). <https://doi.org/10.4121/uuid:a07386a5-7be3-4367-9535-70bc9e77dbe6>
3. Casillas, A., Chen, Y., Granderson, J., Lin, G., Chen, Z., Wen, J., Huang, S.: Development of high-fidelity air handling unit fault models for FDD innovation: lessons learned and recommendations. *Journal of Building Performance Simulation* **17**(5), 615–630 (2024). <https://doi.org/10.1080/19401493.2024.2382757>
4. Deng, Q., Chen, Z., Zhu, W., Li, Z., Yuan, Y., Wang, Y.: Adaptive fusion graph convolutional network based interpretable fault diagnosis method for HVAC systems enhanced by unlabeled data. *Energy and Buildings* **324**, 114901 (2024). <https://doi.org/10.1016/j.enbuild.2024.114901>
5. Frank, S., Lin, G., Jin, X., Singla, R., Farthing, A., Granderson, J.: A performance evaluation framework for building fault detection and diagnosis algorithms. *Energy and Buildings* **192**, 84–92 (2019). <https://doi.org/10.1016/j.enbuild.2019.03.010>
6. Granderson, J., Lin, G., Chen, Y., Casillas, A., Wen, J., Chen, Z., Im, P., Huang, S., Ling, J.: A labeled dataset for building HVAC systems operating in faulted and fault-free states. *Scientific Data* **10**, 342 (2023). <https://doi.org/10.1038/s41597-023-02197-w>
7. Granderson, J., Lin, G., Harding, A., Im, P., Chen, Y.: Building fault detection data to aid diagnostic algorithm creation and performance testing. *Scientific Data* **7**, 65 (2020). <https://doi.org/10.1038/s41597-020-0398-6>
8. Kapoor, S., Narayanan, A.: Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* **4**(9), 100804 (2023). <https://doi.org/10.1016/j.patter.2023.100804>
9. Mannhardt, F.: Sepsis cases – event log. 4TU.ResearchData (2016). <https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460>
10. Mukhtar, A., Hadwiger, M., Wotawa, F., Schweiger, G.: Reproducibility of machine learning-based fault detection and diagnosis for HVAC systems in buildings: An empirical study (2025), arXiv preprint 2508.00880; finds two of sixty-five studies shared code links, one of them broken
11. Singh, Y.: What a falsification protocol refuted about process mining for HVAC fault detection (2027), companion paper, submitted to the ERPM workshop at ICPM 2027; in preparation
12. Suriadi, S., Andrews, R., ter Hofstede, A.H.M., Wynn, M.T.: Event log imperfection patterns for process mining: Towards a systematic approach to cleaning event logs. *Information Systems* **64**, 132–150 (2017). <https://doi.org/10.1016/j.is.2016.07.011>
13. Weytjens, H., De Weerd, J.: Creating unbiased public benchmark datasets with data leakage prevention for predictive process monitoring. In: Business Process Management Workshops (BPM 2021). Lecture Notes in Business Information Processing, vol. 436, pp. 18–29. Springer (2022). https://doi.org/10.1007/978-3-030-94343-1_2
14. Yuill, D.P., Braun, J.E.: Evaluating the performance of fault detection and diagnostics protocols applied to air-cooled unitary air-conditioning equipment. *HVAC&R Research* **19**(7), 882–891 (2013). <https://doi.org/10.1080/10789669.2013.808135>